

NEXORA PHASE 4C — TECHNICAL SPECIFICATION

API, Task Distribution, & SDK Infrastructure

Version: 1.0.0 | Date: 2026-06-26

Priority: P1 — ASYNC ARCHITECTURE, NON-BLOCKING REQUESTS

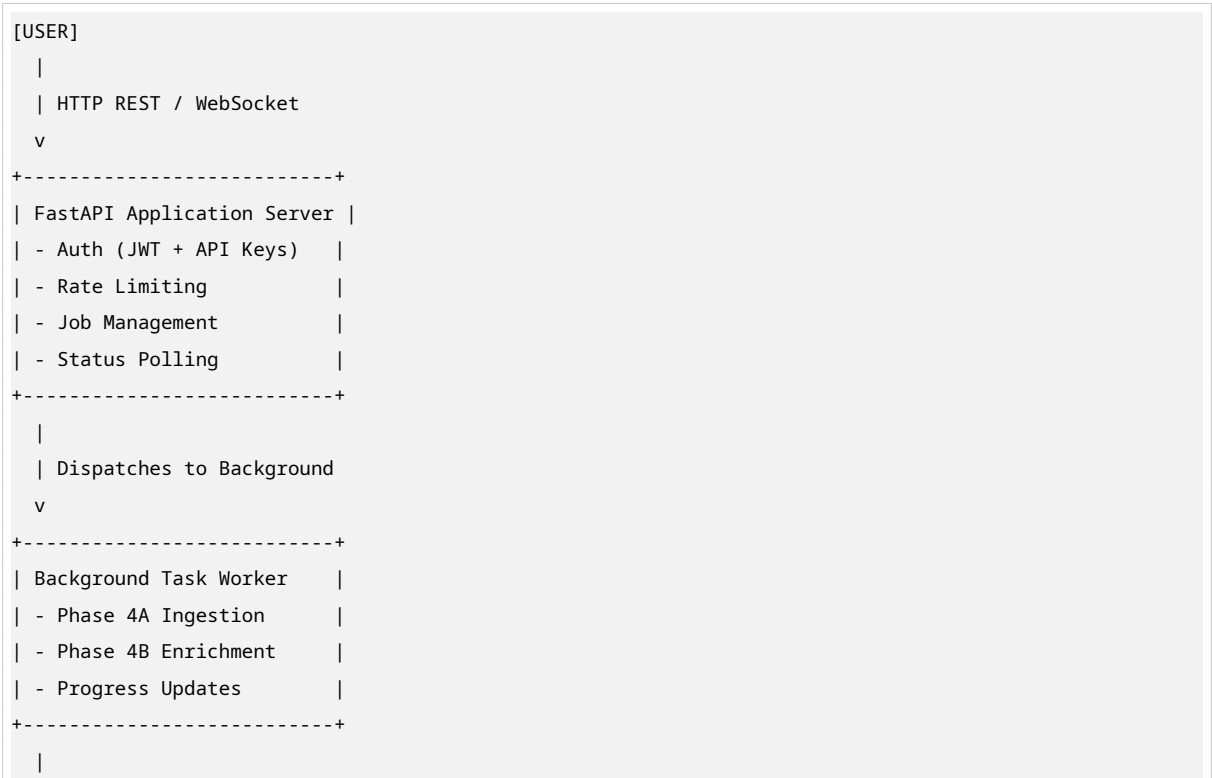
1. ARCHITECTURAL PURPOSE

Phase 4C wraps the lower ingestion and processing engines (4A + 4B) into an **asynchronous backend service layer**. It provides:

- 1. **FastAPI REST API** — HTTP endpoints for crawl jobs, results, and management
- 2. **Background task queue** — Crawls run async, API returns immediately with `job_id`
- 3. **JWT authentication** — Dual mode: JWT for UI sessions, API keys for SDKs
- 4. **Rate limiting** — Per-tenant configurable limits
- 5. **Python CLI** — Command-line tool for developers
- 6. **Python SDK** — Programmatic client library

Core principle: The API server NEVER blocks on crawl execution. It dispatches to background workers and returns `202 Accepted` with a `job_id`.

2. SYSTEM ARCHITECTURE



```
v
[SQLite / ChromaDB / Parquet]
```

3. COMPONENT SPECIFICATIONS

3.1 FastAPI Application Server

File: nexora_crawler/api/server.py

Purpose: Main FastAPI application with lifespan management, middleware, and routers.

3.1.1 Implementation

```
# server.py
# Nexora API Server — Phase 4C
# FastAPI application with JWT auth, rate limiting, OpenAPI docs.
# Start: uvicorn nexora_crawler.api.server:app --host 0.0.0.0 --port 8000

import os
import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

from nexora_crawler.api.routes import auth, crawl, results, admin, health
from nexora_crawler.api.middleware.logging import LoggingMiddleware

logger = logging.getLogger(__name__)

# Rate limiter (in-memory; Redis-backed for distributed)
limiter = Limiter(key_func=get_remote_address)

API_HOST = os.getenv('NEXORA_API_HOST', '0.0.0.0')
API_PORT = int(os.getenv('NEXORA_API_PORT', '8000'))
CORS_ORIGINS = os.getenv('NEXORA_CORS_ORIGINS', '["http://localhost:3000"]')

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Startup and shutdown events."""
    logger.info("[API] Nexora API Server starting on %s:%d", API_HOST, API_PORT)
    logger.info("[API] OpenAPI docs: http://%s:%d/docs", API_HOST, API_PORT)
    logger.info("[API] ReDoc docs: http://%s:%d/redoc", API_HOST, API_PORT)
    yield
    logger.info("[API] Nexora API Server shutting down...")
```

```

app = FastAPI(
    title="Nexora Crawler API",
    description="Production-grade web crawling API with AI enrichment, "
                "Markdown extraction, and batch job management.",
    version="2.0.0",
    lifespan=lifespan,
    docs_url="/docs",
    redoc_url="/redoc",
    openapi_url="/openapi.json",
)

# Rate limiting
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Structured logging
app.add_middleware(LoggingMiddleware)

# Register routers
app.include_router(health.router, tags=["Health"])
app.include_router(auth.router, prefix="/auth", tags=["Authentication"])
app.include_router(crawl.router, prefix="/crawl", tags=["Crawling"])
app.include_router(results.router, prefix="/results", tags=["Results"])
app.include_router(admin.router, prefix="/admin", tags=["Admin"])

@app.get("/")
async def root():
    return {
        "service": "Nexora Crawler API",
        "version": "2.0.0",
        "docs": "/docs",
        "redoc": "/redoc",
        "status": "operational",
    }

```

3.2 Authentication (JWT + API Keys)

File: nexora_crawler/api/routes/auth.py

Purpose: JWT token generation/refresh/validation + API key management.

3.2.1 Implementation

```
# auth.py
# Authentication Routes — Phase 4C
# JWT for UI sessions, API Keys for programmatic access.

import os
import uuid
import hashlib
import secrets
from datetime import datetime, timedelta, timezone

import jwt
import bcrypt
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel, Field

from nexora_crawler.api.database.connection import get_db

router = APIRouter()
security = HTTPBearer()

# JWT Configuration
JWT_SECRET = os.getenv('NEXORA_JWT_SECRET_KEY', 'change-me-in-production')
JWT_ALGORITHM = 'HS256'
JWT_EXPIRE_MINUTES = int(os.getenv('NEXORA_JWT_ACCESS_TOKEN_EXPIRE_MINUTES', '60'))
JWT_REFRESH_EXPIRE_DAYS = int(os.getenv('NEXORA_JWT_REFRESH_TOKEN_EXPIRE_DAYS', '7'))
API_KEY_LENGTH = int(os.getenv('NEXORA_API_KEY_LENGTH', '32'))

# --- Pydantic Models ---

class TokenRequest(BaseModel):
    username: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"
    expires_in: int

class APIKeyResponse(BaseModel):
    api_key: str
    key_name: str
    created_at: str
```

```

class APIKeyCreate(BaseModel):
    key_name: str = Field(default="default", max_length=64)

# --- Token Functions ---

def create_access_token(workspace_id: str, role: str = "user") -> str:
    """Generate JWT access token (short-lived)."""
    expire = datetime.now(timezone.utc) + timedelta(minutes=JWT_EXPIRE_MINUTES)
    payload = {
        "sub": workspace_id,
        "role": role,
        "exp": expire,
        "iat": datetime.now(timezone.utc),
        "type": "access",
    }
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

def create_refresh_token(workspace_id: str) -> str:
    """Generate JWT refresh token (long-lived)."""
    expire = datetime.now(timezone.utc) + timedelta(days=JWT_REFRESH_EXPIRE_DAYS)
    payload = {
        "sub": workspace_id,
        "exp": expire,
        "iat": datetime.now(timezone.utc),
        "type": "refresh",
        "jti": str(uuid.uuid4()),
    }
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)) -> dict:
    """Verify JWT and return payload."""
    try:
        payload = jwt.decode(
            credentials.credentials,
            JWT_SECRET,
            algorithms=[JWT_ALGORITHM],
        )
        if payload.get("type") != "access":
            raise HTTPException(status_code=401, detail="Invalid token type")
        return payload
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

# --- Routes ---

```

```

@router.post("/token", response_model=TokenResponse)
async def login(request: TokenRequest):
    """Authenticate and receive JWT tokens."""
    # In production, verify against database
    # Demo credentials: admin / admin123
    if request.username != "admin" or request.password != "admin123":
        raise HTTPException(status_code=401, detail="Invalid credentials")

    workspace_id = f"ws_{uuid.uuid4().hex[:8]}"

    return TokenResponse(
        access_token=create_access_token(workspace_id, role="admin"),
        refresh_token=create_refresh_token(workspace_id),
        expires_in=JWT_EXPIRE_MINUTES * 60,
    )

@router.post("/refresh", response_model=TokenResponse)
async def refresh_token(refresh_token: str):
    """Refresh an expired access token."""
    try:
        payload = jwt.decode(
            refresh_token,
            JWT_SECRET,
            algorithms=[JWT_ALGORITHM],
        )
        if payload.get("type") != "refresh":
            raise HTTPException(status_code=401, detail="Invalid token type")

        workspace_id = payload.get("sub")
        return TokenResponse(
            access_token=create_access_token(workspace_id),
            refresh_token=create_refresh_token(workspace_id),
            expires_in=JWT_EXPIRE_MINUTES * 60,
        )
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Refresh token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid refresh token")

@router.post("/api-keys", response_model=APIKeyResponse)
async def create_api_key(
    request: APIKeyCreate,
    payload: dict = Depends(verify_token),
):
    """Create a new API key for programmatic access."""
    workspace_id = payload.get("sub")
    api_key = secrets.token_hex(API_KEY_LENGTH)
    api_key_hash = hashlib.sha256(api_key.encode()).hexdigest()

```

```

db = await get_db()
await db.execute(
    "INSERT INTO api_keys (workspace_id, api_key_hash, key_name) VALUES (?, ?, ?)",
    (workspace_id, api_key_hash, request.key_name),
)

return APIKeyResponse(
    api_key=api_key,
    key_name=request.key_name,
    created_at=datetime.now(timezone.utc).isoformat(),
)

@router.get("/me")
async def get_current_user(payload: dict = Depends(verify_token)):
    """Get current user/workspace info."""
    return {
        "workspace_id": payload.get("sub"),
        "role": payload.get("role", "user"),
        "token_type": payload.get("type"),
    }

```

3.3 Crawl Job Management

File: nexora_crawler/api/routes/crawl.py

Purpose: Submit crawl jobs, check status, cancel jobs. All crawls run in background.

3.3.1 Implementation

```

# crawl.py
# Crawl Routes — Phase 4C
# Job submission, status polling, cancellation.
# All crawls dispatched to background tasks.

import uuid
from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, Query, BackgroundTasks
from pydantic import BaseModel, Field, HttpUrl

from nexora_crawler.api.middleware.auth import verify_token
from nexora_crawler.api.tasks.crawl_task import run_crawl_job

router = APIRouter()

# In-memory job store (replace with Redis/DB in production)
_jobs: dict[str, dict] = {}

```

```
# --- Pydantic Models ---

class CrawlRequest(BaseModel):
    url: HttpUrl
    strategy: str = Field(
        default="whole-website",
        pattern=r"^(single-page|linked-pages|whole-website|everything)$",
    )
    max_pages: int = Field(default=100, ge=1, le=100000)
    output_format: str = Field(
        default="json",
        pattern=r"^(json|csv|parquet|markdown)$",
    )
    playwright: bool = Field(default=False)
    javascript: bool = Field(default=True)
```

```
class BatchCrawlRequest(BaseModel):
    urls: List[HttpUrl]
    strategy: str = Field(default="single-page")
    max_pages_per_url: int = Field(default=10, ge=1, le=1000)
    output_format: str = Field(default="json")
```

```
class CrawlResponse(BaseModel):
    job_id: str
    status: str
    message: str
    estimated_time_seconds: int = 30
```

```
class JobStatusResponse(BaseModel):
    job_id: str
    status: str # queued, running, completed, failed, cancelled
    progress: float = 0.0
    pages_crawled: int = 0
    total_pages: int = 0
    current_url: Optional[str] = None
    started_at: Optional[str] = None
    completed_at: Optional[str] = None
    estimated_completion: Optional[str] = None
    error: Optional[str] = None
```

```
# --- Routes ---
```

```
@router.post("/start", response_model=CrawlResponse)
async def start_crawl(
    request: CrawlRequest,
    background_tasks: BackgroundTasks,
    payload: dict = Depends(verify_token),
```



```

):
    """Submit a single URL crawl job. Returns immediately with job_id."""
    workspace_id = payload.get("sub")
    job_id = str(uuid.uuid4())

    # Store job metadata
    _jobs[job_id] = {
        "job_id": job_id,
        "workspace_id": workspace_id,
        "status": "queued",
        "progress": 0.0,
        "pages_crawled": 0,
        "total_pages": request.max_pages,
        "started_at": None,
        "error": None,
    }

    # Dispatch to background task
    background_tasks.add_task(
        run_crawl_job,
        job_id=job_id,
        url=str(request.url),
        strategy=request.strategy,
        max_pages=request.max_pages,
        output_format=request.output_format,
        workspace_id=workspace_id,
    )

    _jobs[job_id]["status"] = "running"
    _jobs[job_id]["started_at"] =
        ↪ __import__('datetime').datetime.now(__import__('datetime').timezone.utc).isoformat()

    return CrawlResponse(
        job_id=job_id,
        status="queued",
        message=f"Crawl job queued for {request.url}",
        estimated_time_seconds=min(request.max_pages * 3, 300),
    )

@router.post("/batch", response_model=List[CrawlResponse])
async def start_batch_crawl(
    request: BatchCrawlRequest,
    background_tasks: BackgroundTasks,
    payload: dict = Depends(verify_token),
):
    """Submit multiple URLs for batch crawling."""
    workspace_id = payload.get("sub")
    responses = []

    for url in request.urls:

```

```

        job_id = str(uuid.uuid4())
        _jobs[job_id] = {
            "job_id": job_id,
            "workspace_id": workspace_id,
            "status": "queued",
            "progress": 0.0,
            "pages_crawled": 0,
            "total_pages": request.max_pages_per_url,
            "started_at": None,
            "error": None,
        }

        background_tasks.add_task(
            run_crawl_job,
            job_id=job_id,
            url=str(url),
            strategy=request.strategy,
            max_pages=request.max_pages_per_url,
            output_format=request.output_format,
            workspace_id=workspace_id,
        )

        _jobs[job_id]["status"] = "running"
        responses.append(CrawlResponse(
            job_id=job_id,
            status="queued",
            message=f"Crawl job queued for {url}",
        ))

    return responses

@router.get("/status/{job_id}", response_model=JobStatusResponse)
async def get_job_status(
    job_id: str,
    payload: dict = Depends(verify_token),
):
    """Get real-time status of a crawl job."""
    if job_id not in _jobs:
        raise HTTPException(status_code=404, detail="Job not found")

    job = _jobs[job_id]
    return JobStatusResponse(
        job_id=job_id,
        status=job["status"],
        progress=job.get("progress", 0.0),
        pages_crawled=job.get("pages_crawled", 0),
        total_pages=job.get("total_pages", 0),
        started_at=job.get("started_at"),
        error=job.get("error"),
    )

```

```

@router.post("/cancel/{job_id}")
async def cancel_job(
    job_id: str,
    payload: dict = Depends(verify_token),
):
    """Cancel a running crawl job."""
    if job_id not in _jobs:
        raise HTTPException(status_code=404, detail="Job not found")

    _jobs[job_id]["status"] = "cancelled"
    return {
        "job_id": job_id,
        "status": "cancelled",
        "message": f"Job {job_id} cancelled",
    }

@router.get("/list")
async def list_jobs(
    limit: int = Query(default=50, le=100),
    status: Optional[str] = Query(default=None),
    payload: dict = Depends(verify_token),
):
    """List crawl jobs for the authenticated workspace."""
    workspace_id = payload.get("sub")
    jobs = [
        job for job in _jobs.values()
        if job.get("workspace_id") == workspace_id
        and (status is None or job.get("status") == status)
    ]
    jobs = jobs[:limit]

    return {
        "workspace_id": workspace_id,
        "total_jobs": len(jobs),
        "jobs": jobs,
        "limit": limit,
    }

```

3.4 Background Task Worker

File: nexora_crawler/api/tasks/crawl_task.py

Purpose: Execute crawl jobs in background, update job status.

3.4.1 Implementation

```

# crawl_task.py
# Background Task Worker — Phase 4C

```

```

# Executes crawl jobs asynchronously, updates progress.

import asyncio
import logging
from datetime import datetime, timezone

from scrapy.crawler import CrawlerProcess
from scrapy.utils.project import get_project_settings

logger = logging.getLogger(__name__)

# Reference to in-memory job store (injected from crawl.py)
_jobs_store = None

def set_jobs_store(store: dict):
    """Inject job store reference."""
    global _jobs_store
    _jobs_store = store

async def run_crawl_job(
    job_id: str,
    url: str,
    strategy: str,
    max_pages: int,
    output_format: str,
    workspace_id: str,
):
    """
    Background task: run a Scrapy crawl and update job status.

    This runs in a separate thread to avoid blocking the event loop.
    """
    if _jobs_store is None:
        logger.error("[CrawlTask] Jobs store not initialized")
        return

    try:
        _jobs_store[job_id]["status"] = "running"

        # Run Scrapy in executor to not block asyncio loop
        loop = asyncio.get_event_loop()
        await loop.run_in_executor(
            None,
            _run_scrapy_crawl,
            job_id, url, strategy, max_pages, output_format,
        )

        _jobs_store[job_id]["status"] = "completed"
        _jobs_store[job_id]["progress"] = 100.0

```

```

        _jobs_store[job_id]["completed_at"] = datetime.now(timezone.utc).isoformat()

        logger.info("[CrawlTask] Job %s completed", job_id)

    except Exception as exc:
        logger.error("[CrawlTask] Job %s failed: %s", job_id, exc)
        _jobs_store[job_id]["status"] = "failed"
        _jobs_store[job_id]["error"] = str(exc)

def _run_scrapy_crawl(
    job_id: str,
    url: str,
    strategy: str,
    max_pages: int,
    output_format: str,
):
    """Synchronous Scrapy execution."""
    settings = get_project_settings()
    settings.set("FEED_FORMAT", output_format if output_format != "markdown" else "json")
    settings.set("JOB_ID", job_id)

    process = CrawlerProcess(settings)
    process.crawl(
        "nexora",
        urls=url,
        strategy=strategy,
        max_pages=max_pages,
    )
    process.start()

```

3.5 Database Connection

File: nexora_crawler/api/database/connection.py

Purpose: Async SQLite/PostgreSQL connection manager.

3.5.1 Implementation

```

# connection.py
# Database Connection Manager — Phase 4C
# Async SQLite (dev) / PostgreSQL (prod)

import os
import logging
from typing import Optional

logger = logging.getLogger(__name__)

DATABASE_URL = os.getenv('NEXORA_DATABASE_URL', 'sqlite:///./nexora.db')

```

```

class DatabaseConnection:
    """Async database connection manager."""

    def __init__(self):
        self._connection = None
        self._url = DATABASE_URL

    async def connect(self):
        if self._url.startswith("sqlite"):
            import aiosqlite
            db_path = self._url.replace("sqlite://", "")
            self._connection = await aiosqlite.connect(db_path)
            self._connection.row_factory = aiosqlite.Row
            logger.info("[DB] Connected to SQLite: %s", db_path)
        else:
            import asyncpg
            self._connection = await asyncpg.connect(self._url)
            logger.info("[DB] Connected to PostgreSQL")

        await self._init_schema()

    async def disconnect(self):
        if self._connection:
            await self._connection.close()
            logger.info("[DB] Disconnected")

    async def _init_schema(self):
        await self.execute("""
            CREATE TABLE IF NOT EXISTS workspaces (
                id TEXT PRIMARY KEY,
                name TEXT NOT NULL,
                created_at TEXT NOT NULL DEFAULT (datetime('now')),
                quota_pages INTEGER DEFAULT 10000,
                quota_storage_gb INTEGER DEFAULT 1
            )
        """)

        await self.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id TEXT PRIMARY KEY,
                workspace_id TEXT NOT NULL,
                username TEXT UNIQUE NOT NULL,
                password_hash TEXT NOT NULL,
                role TEXT DEFAULT 'user',
                created_at TEXT NOT NULL DEFAULT (datetime('now')),
                FOREIGN KEY (workspace_id) REFERENCES workspaces(id)
            )
        """)

```

```

await self.execute("""
    CREATE TABLE IF NOT EXISTS api_keys (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        workspace_id TEXT NOT NULL,
        api_key_hash TEXT NOT NULL,
        key_name TEXT DEFAULT 'default',
        created_at TEXT NOT NULL DEFAULT (datetime('now')),
        last_used_at TEXT,
        FOREIGN KEY (workspace_id) REFERENCES workspaces(id)
    )
""")

logger.info("[DB] Schema initialized")

async def execute(self, query: str, params: tuple = None):
    if not self._connection:
        await self.connect()

    if self._url.startswith("sqlite"):
        cursor = await self._connection.execute(query, params or ())
        await self._connection.commit()
        return cursor
    else:
        return await self._connection.execute(query, *params) if params else await
        ↪ self._connection.execute(query)

async def fetch_one(self, query: str, params: tuple = None):
    if not self._connection:
        await self.connect()

    if self._url.startswith("sqlite"):
        cursor = await self._connection.execute(query, params or ())
        return await cursor.fetchone()
    else:
        return await self._connection.fetchrow(query, *params) if params else await
        ↪ self._connection.fetchrow(query)

async def fetch_all(self, query: str, params: tuple = None):
    if not self._connection:
        await self.connect()

    if self._url.startswith("sqlite"):
        cursor = await self._connection.execute(query, params or ())
        return await cursor.fetchall()
    else:
        return await self._connection.fetch(query, *params) if params else await
        ↪ self._connection.fetch(query)

```

```
_db: Optional[DatabaseConnection] = None
```

```
async def get_db() -> DatabaseConnection:
    global _db
    if _db is None:
        _db = DatabaseConnection()
        await _db.connect()
    return _db
```

3.6 Logging Middleware

File: nexora_crawler/api/middleware/logging.py

Purpose: Structured request/response logging.

3.6.1 Implementation

```
# logging.py
# Logging Middleware — Phase 4C
# Structured request/response logging for API observability.

import time
import logging
from datetime import datetime, timezone

from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware

logger = logging.getLogger("nexora.api")

class LoggingMiddleware(BaseHTTPMiddleware):
    """Log all API requests with structured format."""

    async def dispatch(self, request: Request, call_next):
        start_time = time.time()

        method = request.method
        path = request.url.path
        client_host = request.client.host if request.client else "unknown"

        try:
            response = await call_next(request)
            status_code = response.status_code
        except Exception as exc:
            status_code = 500
            logger.error("[API] Unhandled error: %s", exc)
            raise

        duration_ms = (time.time() - start_time) * 1000
```



```

log_entry = {
    "timestamp": datetime.now(timezone.utc).isoformat(),
    "method": method,
    "path": path,
    "status_code": status_code,
    "duration_ms": round(duration_ms, 2),
    "client_host": client_host,
}

if status_code >= 500:
    logger.error("[API] %s %s -> %d (%dms)", method, path, status_code, duration_ms)
elif status_code >= 400:
    logger.warning("[API] %s %s -> %d (%dms)", method, path, status_code, duration_ms)
else:
    logger.info("[API] %s %s -> %d (%dms)", method, path, status_code, duration_ms)

return response

```

3.7 Health Check

File: nexora_crawler/api/routes/health.py

Purpose: Basic and detailed health endpoints.

3.7.1 Implementation

```

# health.py
# Health Check Routes — Phase 4C

import time
import platform
from datetime import datetime, timezone

from fastapi import APIRouter

router = APIRouter()
start_time = time.time()

@router.get("/health")
async def health_check():
    return {
        "status": "healthy",
        "service": "nexora-api",
        "version": "2.0.0",
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }

@router.get("/health/detailed")

```

```

async def detailed_health():
    uptime_seconds = int(time.time() - start_time)

    return {
        "status": "healthy",
        "uptime": {
            "seconds": uptime_seconds,
            "hours": round(uptime_seconds / 3600, 2),
        },
        "system": {
            "platform": platform.platform(),
            "python_version": platform.python_version(),
        },
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }

```

3.8 CLI Application

File: nexora_crawler/cli/main.py

Purpose: Command-line interface for quick crawling and API interaction.

3.8.1 Implementation

```

# main.py
# Nexora CLI — Phase 4C
# Quick command-line interface for crawling.

import argparse
import sys
import os
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent.parent))

DEFAULT_OUTPUT_DIR = "./nexora_output"

def main():
    parser = argparse.ArgumentParser(
        prog="nexora",
        description="Nexora Web Crawler CLI",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""Examples:
    nexora https://example.com
    nexora https://example.com -o markdown
    nexora https://example.com -s whole-website -m 500
    nexora --api http://localhost:8000 crawl https://example.com""",
    )

```

```

parser.add_argument("url", nargs="?", help="Target URL to crawl")
parser.add_argument("-o", "--output", choices=["json", "csv", "markdown", "parquet"],
                    default="json")
parser.add_argument("-s", "--strategy",
                    choices=["single-page", "linked-pages", "whole-website", "everything"],
                    default="single-page")
parser.add_argument("-m", "--max-pages", type=int, default=100)
parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR)
parser.add_argument("--api", help="Nexora API server URL")
parser.add_argument("--api-key", help="API key for authentication")
parser.add_argument("--quiet", "-q", action="store_true")
parser.add_argument("--version", action="version", version="Nexora 2.0.0")

subparsers = parser.add_subparsers(dest="command")

crawl_parser = subparsers.add_parser("crawl", help="Crawl via API")
crawl_parser.add_argument("url")
crawl_parser.add_argument("-o", "--output", default="json")
crawl_parser.add_argument("-s", "--strategy", default="single-page")
crawl_parser.add_argument("-m", "--max-pages", type=int, default=100)

status_parser = subparsers.add_parser("status", help="Check job status")
status_parser.add_argument("job_id")

list_parser = subparsers.add_parser("list-jobs", help="List recent jobs")
list_parser.add_argument("--limit", type=int, default=10)

args = parser.parse_args()

if not args.url and not args.command:
    parser.print_help()
    return

if args.api:
    _run_api_mode(args)
else:
    _run_direct_mode(args)

def _run_direct_mode(args):
    if not args.url:
        print("Error: URL required")
        sys.exit(1)

    if not args.quiet:
        print(f"\n${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
        ↪ 1f50d.png}}}}}}${ Nexora Crawler v2.0.0")
        print(f"  URL: {args.url}")
        print(f"  Strategy: {args.strategy}")
        print(f"  Max Pages: {args.max_pages}")
        print(f"  Output: {args.output}")

```

```

try:
    from scrapy.crawler import CrawlerProcess
    from scrapy.utils.project import get_project_settings

    settings = get_project_settings()
    settings.set("FEED_FORMAT", args.output if args.output != "markdown" else "json")
    settings.set("FEED_URI", os.path.join(args.output_dir, f"crawl_output.{args.output}"))

    process = CrawlerProcess(settings)
    process.crawl("nexora", urls=args.url, strategy=args.strategy, max_pages=args.max_pages)
    process.start()

    if not args.quiet:
        print(f"\n$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2705.png}}}}}}$$ Crawl complete!")
except Exception as e:
    print(f"$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/274c.png}}}}}}$$ Crawl failed: {e}")
    sys.exit(1)

def _run_api_mode(args):
    import httpx
    base_url = args.api.rstrip("/")
    headers = {}
    if args.api_key:
        headers["Authorization"] = f"Bearer {args.api_key}"

    if args.command == "crawl":
        resp = httpx.post(f"{base_url}/crawl/start", json={
            "url": args.url or args.url,
            "strategy": args.strategy,
            "max_pages": args.max_pages,
            "output_format": args.output,
        }, headers=headers)
        if resp.status_code == 200:
            data = resp.json()
            print(f"$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2705.png}}}}}}$$ Job submitted: {data['job_id']}")
        else:
            print(f"$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/274c.png}}}}}}$$ Error: {resp.status_code} - {resp.text}")

    elif args.command == "status":
        resp = httpx.get(f"{base_url}/crawl/status/{args.job_id}", headers=headers)
        if resp.status_code == 200:
            data = resp.json()
            print(f"$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4ca.png}}}}}}$$ Job: {data['job_id']}")
            print(f"    Status: {data['status']}")

```

```

        print(f"    Progress: {data['progress']:.1f}%")
    else:
        print(f"$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
        ↪ emoji/274c.png}}}}}}$$ Error: {resp.status_code}")

elif args.command == "list-jobs":
    resp = httpx.get(f"{base_url}/crawl/list?limit={args.limit}", headers=headers)
    if resp.status_code == 200:
        data = resp.json()
        print(f"$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
        ↪ emoji/1f4cb.png}}}}}}$$ Jobs: {data['total_jobs']} total")
        for job in data.get("jobs", []):
            print(f"    {job['job_id']} - {job['status']}")
    else:
        print(f"$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
        ↪ emoji/274c.png}}}}}}$$ Error: {resp.status_code}")

if __name__ == "__main__":
    main()

```

3.9 Python SDK

File: nexora_crawler/sdk/client.py

Purpose: Programmatic client for the Nexora API.

3.9.1 Implementation

```

# client.py
# Nexora Python SDK — Phase 4C
# Programmatic API client for developers.

import time
from typing import Dict, List, Optional, Union
from dataclasses import dataclass

import httpx

@dataclass
class CrawlResult:
    job_id: str
    status: str
    message: str
    estimated_time_seconds: int = 30

@dataclass
class JobStatus:
    job_id: str

```

```

status: str
progress: float = 0.0
pages_crawled: int = 0
total_pages: int = 0
current_url: Optional[str] = None
started_at: Optional[str] = None
completed_at: Optional[str] = None
error: Optional[str] = None

class NexoraClient:
    """
    Python SDK for the Nexora Crawler API.

    Usage:
        client = NexoraClient(base_url="http://localhost:8000")
        result = client.crawl("https://example.com")
        status = client.wait_for_completion(result.job_id)
    """

    def __init__(
        self,
        api_key: Optional[str] = None,
        base_url: str = "http://localhost:8000",
        timeout: int = 60,
    ):
        self.base_url = base_url.rstrip("/")
        self.timeout = timeout

        headers = {"Content-Type": "application/json"}
        if api_key:
            headers["Authorization"] = f"Bearer {api_key}"

        self._client = httpx.Client(
            base_url=self.base_url,
            headers=headers,
            timeout=timeout,
        )

    def crawl(
        self,
        url: str,
        strategy: str = "whole-website",
        max_pages: int = 100,
        output_format: str = "json",
        wait: bool = False,
        poll_interval: int = 2,
    ) -> Union[CrawlResult, Dict]:
        """Submit a crawl job. Optionally wait for completion."""
        response = self._client.post("/crawl/start", json={
            "url": url,

```

```

        "strategy": strategy,
        "max_pages": max_pages,
        "output_format": output_format,
    })
    response.raise_for_status()
    data = response.json()
    result = CrawlResult(**data)

    if wait:
        return self.wait_for_completion(result.job_id, poll_interval)
    return result

def batch_crawl(
    self,
    urls: List[str],
    strategy: str = "single-page",
    max_pages_per_url: int = 10,
    output_format: str = "json",
) -> List[CrawlResult]:
    """Submit multiple URLs for batch crawling."""
    response = self._client.post("/crawl/batch", json={
        "urls": urls,
        "strategy": strategy,
        "max_pages_per_url": max_pages_per_url,
        "output_format": output_format,
    })
    response.raise_for_status()
    return [CrawlResult(**item) for item in response.json()]

def get_job_status(self, job_id: str) -> JobStatus:
    """Get real-time status of a crawl job."""
    response = self._client.get(f"/crawl/status/{job_id}")
    response.raise_for_status()
    return JobStatus(**response.json())

def cancel_job(self, job_id: str) -> Dict:
    """Cancel a running crawl job."""
    response = self._client.post(f"/crawl/cancel/{job_id}")
    response.raise_for_status()
    return response.json()

def list_jobs(self, limit: int = 50, status: Optional[str] = None) -> Dict:
    """List crawl jobs."""
    params = {"limit": limit}
    if status:
        params["status"] = status
    response = self._client.get("/crawl/list", params=params)
    response.raise_for_status()
    return response.json()

def wait_for_completion(

```

```

self,
job_id: str,
poll_interval: int = 2,
timeout: Optional[int] = None,
) -> Dict:
    """Poll job status until completion."""
    start_time = time.time()

    while True:
        if timeout and (time.time() - start_time) > timeout:
            raise TimeoutError(f"Job {job_id} did not complete within {timeout}s")

        status = self.get_job_status(job_id)

        if status.status in ("completed", "failed", "cancelled"):
            return {
                "job_id": status.job_id,
                "status": status.status,
                "pages_crawled": status.pages_crawled,
                "total_pages": status.total_pages,
                "error": status.error,
            }

        time.sleep(poll_interval)

def health_check(self) -> Dict:
    """Check API server health."""
    response = self._client.get("/health")
    response.raise_for_status()
    return response.json()

def close(self):
    self._client.close()

def __enter__(self):
    return self

def __exit__(self, *args):
    self.close()

```

4. ENVIRONMENT VARIABLES

```

# API Server
NEXORA_API_HOST=0.0.0.0
NEXORA_API_PORT=8000
NEXORA_API_WORKERS=4
NEXORA_API_LOG_LEVEL=info

# Authentication

```



```
NEXORA_JWT_SECRET_KEY=your-secret-key-change-in-production
NEXORA_JWT_ALGORITHM=HS256
NEXORA_JWT_ACCESS_TOKEN_EXPIRE_MINUTES=60
NEXORA_JWT_REFRESH_TOKEN_EXPIRE_DAYS=7
NEXORA_API_KEY_LENGTH=32

# Rate Limiting
NEXORA_RATE_LIMIT_DEFAULT=60      # requests per minute
NEXORA_RATE_LIMIT_BURST=10

# Database
NEXORA_DATABASE_URL=sqlite:///./nexora.db

# CORS
NEXORA_CORS_ORIGINS=["http://localhost:3000", "http://localhost:1420"]

# Logging
NEXORA_LOG_FORMAT=json
NEXORA_LOG_LEVEL=info
NEXORA_STRUCTURED_LOGS=true
```

5. DEPENDENCIES

```
# API Server
pip install fastapi==0.111.0
pip install uvicorn[standard]==0.29.0
pip install pydantic==2.7.0

# Authentication
pip install PyJWT==2.8.0
pip install bcrypt==4.1.0
pip install python-multipart==0.0.9

# Rate Limiting
pip install slowapi==0.1.9

# CORS & Middleware
pip install httpx==0.27.0

# Async Database
pip install aioredis==2.0.0
# OR: pip install asyncpg==0.29.0 # Production

# Developer Tools
pip install python-dotenv==1.0.1
```

6. TEST MATRIX

Test ID	Scenario	Expected Result
P4C-T01	API health check	GET /health returns 200 with status, version
P4C-T02	JWT login	POST /auth/token returns access + refresh tokens
P4C-T03	JWT validation	Protected endpoint rejects invalid token with 401
P4C-T04	Token refresh	POST /auth/refresh returns new access token
P4C-T05	API key creation	POST /auth/api-keys returns new API key
P4C-T06	Rate limiting	>60 req/min returns 429
P4C-T07	Crawl submission	POST /crawl/start returns 202 with job_id
P4C-T08	Job status polling	GET /crawl/status/{id} returns progress
P4C-T09	Job cancellation	POST /crawl/cancel/{id} stops job
P4C-T10	Batch crawl	POST /crawl/batch returns multiple job_ids
P4C-T11	CLI direct mode	nexora https://example.com runs crawl
P4C-T12	CLI API mode	nexora --api ... crawl ... submits via API
P4C-T13	SDK crawl	client.crawl(url) returns CrawlResult
P4C-T14	SDK wait	client.wait_for_completion(id) polls until done
P4C-T15	OpenAPI docs	/docs and /redoc render correctly
P4C-T16	Non-blocking	API returns immediately, crawl runs in background
P4C-T17	No regression	Phase 3 + 4A + 4B tests still pass

7. DEFINITION OF DONE

- FastAPI server starts and responds to /health
- JWT authentication works (login, refresh, validation)
- Rate limiting enforced per endpoint
- Crawl jobs submitted via POST /crawl/start return 202 with job_id
- Job status polling works via GET /crawl/status/{job_id}
- Background tasks execute crawls without blocking API
- CLI works in direct mode (no API needed)
- CLI works in API mode
- Python SDK installs and works with API
- OpenAPI docs render at /docs and /redoc
- All 17 test cases pass
- Phase 3 + 4A + 4B tests show no regression